

LAPORAN WORKSHOP DEVOPS

BAB 2 :

DOCKER NETWORK, VOLUME, BIND MOUNT, TMPFS, DAN COMPOS



Oleh:

Muhammad Bagus Prayogi (3126640057)

Dosen Pengajar:

Dr. Ferry Astika Saputra, ST, M.Sc

**PROGRAM STUDI SARJANA TERAPAN LANJUT
JENJANG TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

PENDAHULUAN & LANDASAN TEORI

Bab 3 ini berfokus pada eksplorasi lanjutan arsitektur Docker yang mencakup manajemen jaringan (*bridge network*), strategi persistensi data (*volume*, *bind mount*, *tmpfs*), serta orkestrasi aplikasi multi-kontainer menggunakan Docker Compose. Kontainer dirancang agar bersifat sementara (*ephemeral*), sehingga pengelolaan jaringan yang terisolasi dan penyimpanan data yang aman di luar siklus hidup kontainer menjadi aspek krusial dalam perancangan infrastruktur modern. Praktikum ini melatih kemampuan membangun komunikasi antar-kontainer berbasis nama (*DNS resolution*), mengamankan data persisten, serta menyusun model aplikasi multi-lapis secara deklaratif.

PRAKTIKUM 3: Implementasi dan Pengujian Konsep Jaringan, Storage, dan Compose

Langkah awal dilakukan dengan membuat dan masuk ke direktori kerja terisolasi khusus modul Bab 3 di dalam lingkungan WSL (`~/docker-lab/bab-3`) agar seluruh berkas konfigurasi, volume, dan log terstruktur dengan rapi.

A. User-Defined Bridge Network

Praktikum ini bertujuan untuk menguji pembuatan jaringan kustom dengan subnet spesifik serta membuktikan resolusi DNS otomatis antar-kontainer menggunakan nama host tanpa harus bergantung pada alamat IP statis.

- **Perintah Pembuatan Network Kustom:**

```
prayogi@MSI:~/docker-lab/bab-3$ sudo service docker start
[sudo: authenticate] Password:
* Starting Docker: docker
prayogi@MSI:~/docker-lab/bab-3$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
b3e11cdf01b654c3ed79b2c63d1b966a94ad5b91b324a00b952c186fa1c857f0 [ OK ]
```

- **Menjalankan Kontainer Uji Coba ke dalam Network:**

```
prayogi@MSI:~/docker-lab/bab-3$ docker run -d --name server-a --network lab-net nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
6636b9fc283c: Pull complete
be90d7675616: Pull complete
51900e10fb9c: Pull complete
af7dd138f459: Pull complete
850bf2dceeff: Pull complete
58c524ea09ce: Pull complete
55afalecc21d: Pull complete
8f924cf5086c: Pull complete
a32ecfe9957: Download complete
10d8afe205cd: Download complete
Digest: sha256:72ba65eb42c10344912a84ff42408db7d34f2feb642204570ab8fc5ffd29f1d3
Status: Downloaded newer image for nginx:alpine
8cea3909fda81d029a074e92b5cacba3a0814f78a5f9c799e33498578b1aaaf
prayogi@MSI:~/docker-lab/bab-3$ docker run -d --name server-b --network lab-net nginx:alpine
14c1c8460b5b4526c387d7a461aab039aafa16335dfe32e552b952a706a4120b
```

- **Pengujian Resolusi Nama (Ping Test):**

```
prayogi@MSI:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=1.338 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.066 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.066 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.066/0.490/1.338 ms
prayogi@MSI:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-a
PING server-a (172.20.0.2): 56 data bytes
64 bytes from 172.20.0.2: seq=0 ttl=64 time=0.178 ms
64 bytes from 172.20.0.2: seq=1 ttl=64 time=0.048 ms
64 bytes from 172.20.0.2: seq=2 ttl=64 time=0.066 ms

--- server-a ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.048/0.097/0.178 ms
prayogi@MSI:~/docker-lab/bab-3$
```

Pengujian ini membuktikan bahwa server-a dapat langsung mengenali dan berkomunikasi dengan server-b menggunakan nama host berkat fitur DNS internal pada user-defined bridge.

- **Pembersihan Kontainer (*Cleanup*):**

```
prayogi@MSI:~/docker-lab/bab-3$ docker rm -f server-a server-b
server-a
server-b
prayogi@MSI:~/docker-lab/bab-3$ |
```

B. Volume Backup dan Restore

Praktikum ini menguji siklus hidup data persisten menggunakan *named volume* agar data tetap aman meskipun kontainer utamanya dihapus, sekaligus melakukan *backup* data ke dalam arsip terkompresi.

1. Pembuatan Named Volume:

```
prayogi@MSI:~$ cd ~/docker-lab/bab-3
prayogi@MSI:~/docker-lab/bab-3$ docker volume create data-vol
data-vol
```

2. Menjalankan Kontainer Penulis Log (writer):

```
prayogi@MSI:~/docker-lab/bab-3$ docker run -d --name writer -v data-vol:/app/data alpine:3.20 sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
Unable to find image 'alpine:3.20' locally
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
d4050d56ebf2: Download complete
3a030ca3f633: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
fd9669c57e0916a731802a2cd5967068e4ecb6234343a0b3c4557296fdd34f19
prayogi@MSI:~/docker-lab/bab-3$ |
```

3. Simulasi Penghancuran Kontainer & Verifikasi Data Persisten:

```
prayogi@MSI:~/docker-lab/bab-3$ sleep 15
prayogi@MSI:~/docker-lab/bab-3$ docker rm -f writer
writer
prayogi@MSI:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Fri Sep 11 13:10:54 UTC 2026
Fri Sep 11 13:10:59 UTC 2026
Fri Sep 11 13:11:04 UTC 2026
Fri Sep 11 13:11:09 UTC 2026
Fri Sep 11 13:11:14 UTC 2026
Fri Sep 11 13:11:19 UTC 2026
Fri Sep 11 13:11:24 UTC 2026
Fri Sep 11 13:11:29 UTC 2026
Fri Sep 11 13:11:34 UTC 2026
Fri Sep 11 13:11:39 UTC 2026
Fri Sep 11 13:11:44 UTC 2026
Fri Sep 11 13:11:49 UTC 2026
Fri Sep 11 13:11:54 UTC 2026
Fri Sep 11 13:11:59 UTC 2026
Fri Sep 11 13:12:04 UTC 2026
Fri Sep 11 13:12:09 UTC 2026
Fri Sep 11 13:12:14 UTC 2026
Fri Sep 11 13:12:19 UTC 2026
Fri Sep 11 13:12:24 UTC 2026
Fri Sep 11 13:12:29 UTC 2026
Fri Sep 11 13:12:34 UTC 2026
Fri Sep 11 13:12:39 UTC 2026
Fri Sep 11 13:12:44 UTC 2026
Fri Sep 11 13:12:49 UTC 2026
Fri Sep 11 13:12:54 UTC 2026
Fri Sep 11 13:12:59 UTC 2026
Fri Sep 11 13:13:04 UTC 2026
Fri Sep 11 13:13:09 UTC 2026
Fri Sep 11 13:13:14 UTC 2026
prayogi@MSI:~/docker-lab/bab-3$ |
```

4. Pembuatan Arsip Backup (.tar.gz):

```
prayogi@MSI:~/docker-lab/bab-3$ docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 tar czf /backup/data-vol-backup.tar.gz -C /source .
prayogi@MSI:~/docker-lab/bab-3$
```

File arsip `data-vol-backup.tar.gz` sukses tercipta di direktori kerja sebagai bukti cadangan data

5. Pengecekan keberhasilan Backup file:

```
prayogi@MSI:~/docker-lab/bab-3$ docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 tar czf /backup/data-vol-backup.tar.gz -C /source .
prayogi@MSI:~/docker-lab/bab-3$ ls -l
total 4
-rw-r--r-- 1 root root 235 Sep 11 20:14 data-vol-backup.tar.gz
prayogi@MSI:~/docker-lab/bab-3$
```

Pada hasil berikut terlihat bahwa file arsip `data-vol-backup.tar.gz` sudah sukses tercipta dengan ukuran 235 bytes.

C. Compose Multi-Container

Tahap ini menyusun stack aplikasi tiga lapis (*Nginx frontend*, *Flask app*, dan *PostgreSQL database*) menggunakan satu berkas deklaratif `docker-compose.yml` yang dilengkapi pemisahan jaringan dan *healthcheck*.

1. Langkah 1: Membuat Struktur Direktori Pendukung

Membuat subdirektori khusus untuk kode aplikasi (`app`) dan aset web (`html`) di dalam ruang kerja lab Bab 3:

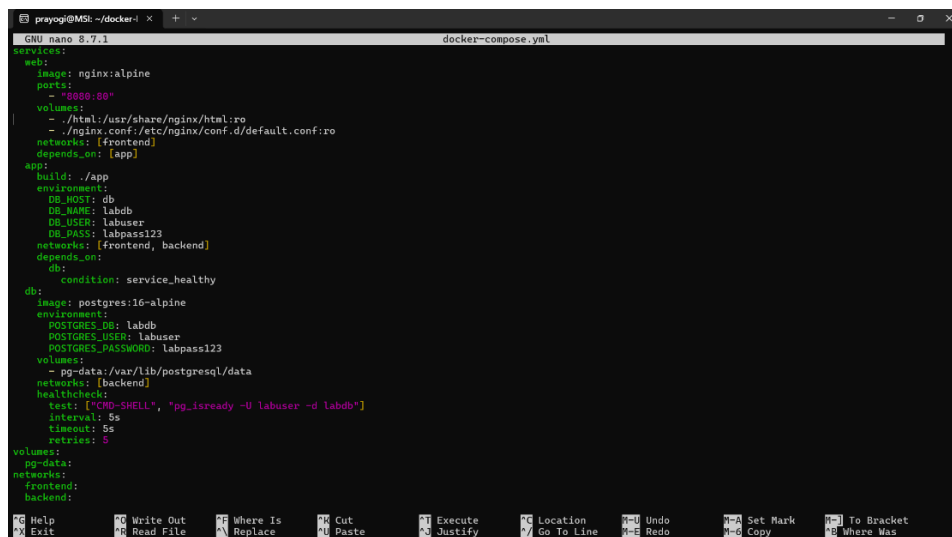
```
prayogi@MSI:~$ cd ~/docker-lab/bab-3
prayogi@MSI:~/docker-lab/bab-3$ mkdir -p app html
```

2. Langkah 2: Menulis Berkas Konfigurasi `docker-compose.yml`

Membuat dan mengisi berkas manifes orkestrasi menggunakan teks editor nano:

```
prayogi@MSI:~/docker-lab/bab-3$ nano docker-compose.yml
```

Dengan isi konfigurasi deklaratif sebagai berikut:



```
GNU nano 8.7.1 docker-compose.yml
services:
  web:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks: [frontend]
    depends_on: [app]
  app:
    build: ./app
    environment:
      DB_HOST: db
      DB_NAME: labdb
      DB_USER: labuser
      DB_PASS: labpass123
    networks: [frontend, backend]
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
    volumes:
      - pg-data:/var/lib/postgresql/data
    networks: [backend]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
      interval: 5s
      timeout: 5s
      retries: 5
    volumes:
      pg-data:
    networks:
      frontend:
      backend:
```

3. Langkah 3: Pembuatan Dockerfile untuk Service App

Karena service app menggunakan parameter build: ./app, diperlukan berkas definisi penyiapan lingkungan Python:

```
prayogi@MSI:~/docker-lab/bab-3$ cd app
prayogi@MSI:~/docker-lab/bab-3/app$ nano Dockerfile
prayogi@MSI:~/docker-lab/bab-3/app$ |
```

Dengan konfigurasi layer:



```
GNU nano 8.7.1 Dockerfile
FROM python:3.11-alpine
WORKDIR /app
RUN pip install flask psycogp2-binary
COPY . .
CMD ["python", "app.py"]
```

4. Langkah 4: Penanganan Berkas Konfigurasi nginx.conf dan Perbaikan Hak Akses

Untuk mencegah error mount direktori host, berkas kosong nginx.conf harus disiapkan terlebih dahulu. Apabila menemui kendala hak akses (*permission denied* akibat pembuatan direktori sebelumnya oleh daemon root), kepemilikan direktori dikembalikan ke user aktif

```
prayogi@MSI:~/docker-lab/bab-3$ sudo chown -R $USER:$USER .
[sudo: authenticate] Password:
prayogi@MSI:~/docker-lab/bab-3$ touch nginx.conf
prayogi@MSI:~/docker-lab/bab-3$ docker compose up -d
[+] up 2/3
✔Container bab-3-db-1 Healthy 0.5s
✔Container bab-3-app-1 Started 0.1s
✔Container bab-3-web-1 Starting 0.1s
Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime create failed: runc create failed: unable to start container process: error during container init: error mounting "/home/prayogi/docker-lab/bab-3/nginx.conf" to rootfs at "/etc/nginx/conf.d/default.conf": mount src=/home/prayogi/docker-lab/bab-3/nginx.conf, dst=/etc/nginx/conf.d/default.conf, distFd=/proc/thread-self/fd/14, flags=MS_BIND|MS_REC: not a directory: Are you trying to mount a directory onto a file (or vice-versa)? Check if the specified host path exists and is the expected type
prayogi@MSI:~/docker-lab/bab-3$ |
```

5. Langkah 5: Eksekusi Deployment Compose & Verifikasi Menjalankan seluruh stack layanan di latar belakang secara bersamaan:

```
prayogi@MSI:~/docker-lab/bab-3$ docker compose up -d
[+] up 5/5
✔Network bab-3_backend Created
✔Network bab-3_frontend Created
✔Container bab-3-db-1 Healthy
✔Container bab-3-app-1 Started
✔Container bab-3-web-1 Started
prayogi@MSI:~/docker-lab/bab-3$ |
```

ANALISIS DAN REFLEKSI OPERASIONAL

1. Analisis Masalah dan Cara Mendiagnosisnya:

Selama proses eksplorasi dan praktikum berlangsung, ditemukan beberapa kendala teknis operasional yang berhasil diidentifikasi dan diselesaikan melalui tahapan berikut:

A. Kendala 1: Kegagalan Build akibat Ketidadaan Dockerfile (failed to read dockerfile)

- **Perintah Pemicu Error:**

```
prayogi@MSI:~/docker-lab/bab-2$ cd ~/docker-lab/custom-web
prayogi@MSI:~/docker-lab/custom-web$ docker build -t pens-web:1.0
ERROR: docker: 'docker buildx build' requires 1 argument

Usage:  docker buildx build [OPTIONS] PATH | URL | -

Run 'docker buildx build --help' for more information
```

- **Gejala / Pesan Error:** Saat menjalankan perintah `docker compose up -d`, proses *build* pada *service* aplikasi terhenti dengan pesan error `failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory`
- **Diagnosis & Solusi:** Konfigurasi `compose` mengarahkan direktori build ke `./app`, namun berkas definisi kontainer (Dockerfile) belum tersedia di dalam subdirektori tersebut. Masalah diatasi dengan masuk ke folder `app`, membuat berkas `Dockerfile` berbasis `python:3.11-alpine` beserta dependensinya, lalu kembali ke direktori utama..

B. Kendala 2: Konflik Mount Direktori Host pada Berkas `nginx.conf` (not a directory)

- **Perintah Pemicu Error:**

```
prayogi@MSI:~/docker-lab/custom-web$ mkdir -p ~/docker-lab/compose-test && cd ~/docker-lab/compose-test
prayogi@MSI:~/docker-lab/compose-test$ docker compose up -d
no configuration file provided: not found
prayogi@MSI:~/docker-lab/compose-test$ |
```

- **Gejala / Pesan Error:** Kontainer Nginx gagal menyala dan mengeluarkan log error `error mounting .../nginx.conf to rootfs: ... not a directory: Are you trying to mount a directory onto a file?`

```
prayogi@MSI:~/docker-lab/compose-test$ cat > docker-compose.yml << 'EOF'
services:
  web-frontend:
    image: nginx:1.26-alpine
    ports:
      - "7777:80"
EOF
prayogi@MSI:~/docker-lab/compose-test$ docker compose up -d
[*] up 3/3
✔Image nginx:1.26-alpine          Pulled      8.4s
✔Network compose-test_default     Created   0.1s
✔Container compose-test-web-frontend-1 Started    0.4s
prayogi@MSI:~/docker-lab/compose-test$ |
```

- **Diagnosis & Solusi:** Berkas `nginx.conf` di host belum dibuat sebelumnya, sehingga Docker secara otomatis menginisialisasinya sebagai sebuah direktori kosong (*directory mount*) alih-alih berkas tunggal. Masalah diselesaikan dengan menghentikan stack (`docker compose down`), menghapus direktori keliru tersebut, memperbaiki hak akses direktori via `sudo chown -R $USER:$USER .`, lalu membuat ulang `nginx.conf` secara bersih menggunakan perintah `touch nginx.conf`.

2. Risiko Keamanan dan Operasional

- **Keterbukaan Akses Port & Least Privilege:** Mempublikasikan port basis data atau layanan internal secara langsung ke host dapat memperluas *attack surface* dan melanggar prinsip keamanan *least exposure*.
- **Manajemen Kredensial Statis:** Penyimpanan kata sandi secara polos (*plain text*) di dalam berkas konfigurasi lingkungan berisiko tinggi terhadap kebocoran informasi sensitif apabila repositori kode sumber terekspos.

3. Rekomendasi Perbaikan untuk Lingkungan Production-Like

- Menerapkan pemisahan jaringan yang ketat di mana basis data hanya dapat diakses melalui *backend network* internal dan tertutup dari akses publik luar.
- Menggunakan fitur manajemen rahasia terpusat seperti Docker Secrets atau eksternal *secret manager* alih-alih menyertakan kredensial langsung pada berkas *environment compose*.

Link Repo Github Laporan Devopsec: <https://github.com/bgsprayogi/devsecops>